



PDF Download
3634737.3656285.pdf
25 December 2025
Total Citations: 2
Total Downloads: 489

Latest updates: <https://dl.acm.org/doi/10.1145/3634737.3656285>

RESEARCH-ARTICLE

HQsFL: A Novel Training Strategy for Constructing High-performance and Quantum-safe Federated Learning

BO YU

HUAJIE SHEN

QIAN XU

WEI HE

WANKUI MAO

QING ZHANG

[View all](#)

Open Access Support provided by:

[School of Cyber Science and Technology, Zhejiang University](#)

Published: 01 July 2024

[Citation in BibTeX format](#)

ASIA CCS '24: 19th ACM Asia
Conference on Computer and
Communications Security
July 1 - 5, 2024
Singapore, Singapore

Conference Sponsors:
[SIGSAC](#)

HQsFL: A Novel Training Strategy for Constructing High-performance and Quantum-safe Federated Learning

Bo Yu ^{*1}	Huajie Shen ¹	Qian Xu	Wei He
Blockchain Research Institute, Bestpay Co., Ltd. Shanghai, China yubo@bestpay.com.cn	Blockchain Research Institute, Bestpay Co., Ltd. Shanghai, China shenhuajie@bestpay.com.cn	Blockchain Research Institute, Bestpay Co., Ltd. Shanghai, China xuqian@bestpay.com.cn	Blockchain Research Institute, Bestpay Co., Ltd. Shanghai, China hewei@bestpay.com.cn

Wankui Mao	Qing Zhang	Fan Zhang
Blockchain Research Institute, Bestpay Co., Ltd. Shanghai, China maowankui@bestpay.com.cn	Blockchain Research Institute, Bestpay Co., Ltd. Shanghai, China zhangqing@bestpay.com.cn	School of Cyber Science and Technology, College of Computer Science and Technology, Zhejiang University Hangzhou, Zhejiang China fanzhang@zju.edu.cn

ABSTRACT

Federated Learning (FL) has attracted increasing attention from both academia and industry due to its merit of securely constructing AI models across multiple entities while preserving the privacy of local training data. However, recent research shows two persisting problems in FL that have yet to be solved: (1) limited practical adaptation of federated learning because of time-consuming conventional privacy-preserving methods, and (2) the absence of quantum-computing resistance in these methods. To address these problems, we propose a novel vertical federated learning strategy, HQsFL, which relies on Fully Homomorphic Encryption (FHE) and Matrix Vector Product basing on Coefficient Encoding. The proposed method can be widely applied to FL algorithms such as logistic regression and XGBoost, etc. We fully implement our approach and evaluate its utility and efficiency through extensive experiments performed on four synthetic datasets. The experimental results demonstrate that our proposed methods for vertical LR and XGBoost achieve comparable levels of AUC to conventional methods, while significantly improving training efficiency and achieving security property of quantum-computing resistance.

CCS CONCEPTS

• Security and privacy → Cryptography; • Computing methodologies → Machine learning.

^{*}Bo Yu is the corresponding author. ¹Bo Yu and ¹Huajie Shen contributed equally to this research.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from [permissions@acm.org](https://permissions.acm.org).

ASIA CCS '24, July 1–5, 2024, Singapore, Singapore

© 2024 Copyright held by the owner/author(s). Publication rights licensed to ACM.

ACM ISBN 979-8-4007-0482-6/24/07...\$15.00

<https://doi.org/10.1145/3634737.3656285>

KEYWORDS

federated learning, fully homomorphic encryption, matrix vector multiplication, quantum-safe cryptography

ACM Reference Format:

Bo Yu, Huajie Shen, Qian Xu, Wei He, Wankui Mao, Qing Zhang, and Fan Zhang. 2024. HQsFL: A Novel Training Strategy for Constructing High-performance and Quantum-safe Federated Learning. In *ACM Asia Conference on Computer and Communications Security (ASIA CCS '24)*, July 1–5, 2024, Singapore, Singapore. ACM, New York, NY, USA, 10 pages. <https://doi.org/10.1145/3634737.3656285>

1 INTRODUCTION

1.1 Background and Motivation

In recent years, the increasing availability of data has fueled the development and adoption of machine learning techniques across various domains. However, the traditional centralized approach to machine learning, which requires pooling and sharing of sensitive data, raises significant privacy concerns and legal constraints (EU's General Data Protection Regulation - GDPR, etc.), particularly when dealing with personal or sensitive information. To cope with these challenges, federated learning initially proposed by McMahan et al. [26] has emerged as a promising paradigm that enables collaborative machine learning without sharing raw data. Instead, the learning process takes place locally on individual devices or servers, with only model updates being exchanged among the participating parties. This decentralized approach ensures individual's data privacy and reduces the risks of data breaching and unauthorized access. However, federated learning still faces some challenges [22, 24], such as how to handle heterogeneous and non-independent data sources, how to deal with malicious or unreliable parties, and how to ensure the correctness and fairness of the global model.

Moreover, with the development of quantum computers [30], crypto security has become a significant concern in the upcoming quantum-computing era. Owing to the special operations on quantum states in superposition, quantum computer is able to efficiently

solve certain math puzzles (such as integer factorization or discrete logarithm that most modern public key cryptography relies on) that the super computers cannot solve in polynomial time. Since the public key crypto algorithms widely exist in FL, threats from quantum computers must be addressed for ensuring the data and system security of data in FL [28, 40]. For instance, FL typically uses the paillier [27] to encrypt the intermediate data, which is aimed at protecting data from being tampered with or stolen. Nevertheless, a malicious attacker equipped with a quantum computer could undermine the security of FL by attacking the paillier algorithm to gain access to plaintext, leading to severe privacy and model attacks. Therefore, to counter the increasing quantum threat, we should optimize algorithm protocols that employ post-quantum secure design to ensure that quantum computing algorithms (such as Shor's algorithm [32], etc.) cannot efficiently solve the cryptographic problems.

1.2 Contributions

We summarize the contributions of this paper as follows:

- We constructed a novel efficient vertical federated logistic regression protocol based on Yang et al. [38], which improves the overall training efficiency by 40x or more and further achieves the property of quantum security. Our proposed method uses coefficient encoding for RLWE based fully homomorphic encryption method to enhance the privacy-preserving and efficiency.
- We proposed an efficient vertical federated XGBoost based on fully homomorphic encryption. Notably, we introduced a novel methodology for distributed histogram construction in the vertical federated XGBoost, incorporating fully homomorphic encryption.
- We analyzed the security of the proposed algorithms and provided research directions to address the problem of quantum security with fully HE.
- A thorough set of experiments demonstrated that our proposed methods for vertical LR and XGBoost achieve the same level of AUC with conventional methods while substantially boosting training efficiency and achieving quantum security.

2 RELATED WORK

As a key technology to protect data privacy, federated learning has recently been developed fast in both academia and industry. Notable works in this area include the application of differential privacy to federated learning frameworks such as federated averaging [26] and secure aggregation protocols [5]. However, the trade-off between privacy and computing efficiency remains a challenge. Up to now, quite a few scholars have proposed some state-of-the-art methods to balance privacy and computing efficiency. One of the most popular approaches is to use homomorphic encryption (HE), which allows for computations to be performed directly on encrypted data. Yang et al. [38] proposed a framework for federated logistic learning that leverages homomorphic encryption for linear models. Chen et al. [9] provided a practical and effective approximation workflow for converting transformer based models into a function that consists of fully HE (FHE) operations. [2] first demonstrated how to train a logistic regression model in a privacy-preserving manner using an approximate encrypted gradient descent approach. However,

all these approaches are applied in horizontal federated learning and the application of FHE to XGBoost [10] remains a challenge. Another approach is to use secure multi-party computation (MPC) protocols, which enable the computation of functions on private data without revealing the data itself Fan et al. [15] investigated MPC-based FL algorithms and introduced a novel approach for two-party vertical federated linear models via an efficient secret sharing (SS) scheme with a trusted coordinator. Dong et al. [13] showed an MPC framework PUMA to enable fast and secure Transformer model inference. However, it can not be applied in actual scenarios due to low efficiency. Chen et al. [8] presented CAESAR, which combines HE and SS to build a secure large-scale sparse logistic regression model and achieves both efficiency and security. Shi et al. [31] proposed a privacy preserving scheme for federated learning by combining the homomorphism of both secret sharing and encryption.

Quantum Federated Learning (QFL) is an emerging area with several studies in recent years [23]. Most QFL works focus on optimization [21], security [28, 34, 40] and implementation [29]. In a different context, secure blockchain-based FL was proposed in [35]. However, quantum-safe FL is poorly studied in the literature as far as we know. A malicious attacker equipped with a quantum computer could undermine the security of FL by attacking the paillier [27] algorithm to gain access to plaintext. Therefore, to counter the increasing quantum threat, we need to design new FL algorithm protocols to ensure post-quantum security, especially for those complex federated learning algorithms (XGBoost, LLM, etc.).

3 PRELIMINARIES

3.1 Notations

We use the notation $\mathcal{R}$ to represent the set of real numbers. A polynomial in $\mathbb{R}_{N,q} = \mathbb{Z}_q[X]/X^N + 1$ is denoted by $\mathbf{a} \in \mathbb{R}_{N,q}$, where $\mathbf{a}[j]$ signifies the j -th coefficient of the polynomial $\mathbf{a}$. We write vectors as lower-case bold symbols, for instance, $\mathbf{v} \in \mathbb{Z}^n$, and matrices as upper-case bold symbols, such as $\mathbf{W} \in \mathbb{Z}^{d \times n}$. The term CT is used to represent a ciphertext, and specifically, $CT(m)$ is used to denote the ciphertext resulting from the encryption of the message m .

3.2 Building blocks

In this paper, we primarily focus on methods pertaining to vertical federated learning (VFL), where multiple parties hold different features of the same sample instances. To ensure clarity of the context, we reference the concepts of the Active Party and Passive Party as proposed by Yang et al. [37] through the following definition. Besides, in our security definition, all parties are honest-but-curious. Some corrupt parties might cooperate with each other in order to gather private information. We provide a concise description of the principal components of classic machine learning, vertical federated learning, and coefficient encoding in the following sections.

Definition:

- **Active Party (AP):** The party that holds the class label, may also hold the feature data.
- **Passive Party (PP):** The party that only holds the feature data.

3.3 Conventional Vertical LR

Logistic Regression(LR) is a machine learning model that is widely employed. Consider $D = \{(x_1, y_1), (x_2, y_2), \dots, (x_n, y_n)\}$ containing n records, where $x_i \in R^d$ is the feature of the i -th sample with d denoting the feature size, and $y_i \in \{0, 1\}$ is its corresponding label. Define the following loss function L with parameters $\theta = (\theta_1, \theta_2, \dots, \theta_d) \in R^d$:

$$L(\theta) = \frac{1}{n} \sum_{i=1}^n [-y_i \log(h_\theta(x_i)) - (1 - y_i) \log(1 - h_\theta(x_i))] + \frac{\lambda}{2m} \sum_{j=1}^d \theta_j^2 \quad (1)$$

where $h_\theta(x)$ is a sigmoid function given by:

$$h_\theta(x) = \frac{1}{1 + \exp(-\theta^T x)} = \frac{1}{1 + \exp(-\sum_{j=1}^d \theta_j x_j)} \quad (2)$$

The training phase of logistic regression aims at finding the minimizer which optimizes the loss function $L(\theta)$, we can use gradient descent to update the model:

$$\theta \leftarrow \theta - \alpha \frac{\partial L}{\partial \theta} \quad (3)$$

where α is the learning rate.

In the context of vertical federated logistic regression, we assume there are two parties: Active Party A and Passive Party B, holding $D_A = \{x_i^A, y_i\}$ and $D_B = \{x_i^B\}$ respectively. As outlined in Algorithm 1, Yang et al. [38] proposed a framework for vertical federated logistic learning based on partially homomorphic encryption (PHE) for linear models, which is also the basis of our proposed method.

3.4 XGBoost and SecureBoost

Extreme Gradient Boosting (XGBoost) [10] is an additive ensemble model that uses a gradient boosting tree. It applies forward stagewise regression, progressively learning new trees to fit the residuals until a termination condition is satisfied. Consider a training dataset $D = \{(x_1, y_1), (x_2, y_2), \dots, (x_n, y_n)\}$ containing n records, where $x_i \in R^d$ is the feature of the i_{th} sample with d denoting the feature size, and y_i is its corresponding label. The final prediction is calculated by summing predictions of all T trees with a fixed learning rate α :

$$\hat{y}_i = \sum_{t=1}^T \alpha f_t(x_i) \quad (4)$$

where T denotes the number of trained decision trees, $f_t(x_i)$ is the prediction made by the t_{th} tree.

The objective of XGBoost is to minimize the sum of the loss of all the data samples. It first calculates the first-order gradient g_i , and second-order hessian h_i of all samples:

$$g_i = \partial_{\hat{y}_i^{(t-1)}} L(y_i, \hat{y}_i^{(t-1)}) \quad (5)$$

$$h_i = \partial_{\hat{y}_i^{(t-1)}}^2 L(y_i, \hat{y}_i^{(t-1)})$$

Where $\hat{y}_i^{(t-1)}$ is the prediction result aggregated from previous trees, y_i is the true label, and $L(y_i, \hat{y}_i^{(t-1)})$ is the loss function.

Algorithm 1 Vertical Federated Logistic Regression [38]

Input: X_A, y in active party A; X_B in passive party B; learning rate α ; paillier key length l ; max iteration T .

Output: θ_A for A; θ_B for B.

- 1: Initial θ_A and θ_B for A and B respectively.
- 2: **for** $t = 1$ **to** T **do**
- 3: A computes $\theta_A^T X_A$; B computes $\theta_B^T X_B$ and sends it to A.
- 4: A computes $\theta^T X = \theta_A^T X_A + \theta_B^T X_B$, $\hat{y} = \text{sigmoid}(\theta^T X)$, $z = y - \hat{y}$, computes loss L .
- 5: A generates paillier keys (pk, sk) , and sends $CT(z) = \text{Paillier.encrypt}(pk, z)$ to B.
- 6: A computes local plaintext gradients $g^A = z^T X_A$;
- 7: B computes local encrypted gradients $CT(g^B) = CT(z)^T X_B$, generates random perturbation R , and sends $CT(g^B + R) = CT(g^B) + R$ to A.
- 8: A decrypts $(g^B + R) = \text{Paillier.decrypt}(sk, CT(g^B + R))$ and sends it to B.
- 9: B uses $g^B + R$ to minus R , gets g^B .
- 10: A and B use gradient descent to update $\theta_A \leftarrow \theta_A - \alpha g^A$ and $\theta_B \leftarrow \theta_B - \alpha g^B$ respectively.
- 11: **end for**
- 12: **Return** θ_A for A; θ_B for B.

The binary cross entropy loss function is typically used as a loss function. Then the gradient sums of instance set I_j on each node j can be calculated by:

$$G_j = \sum_{i \in I_j} g_i$$

$$H_j = \sum_{i \in I_j} h_i \quad (6)$$

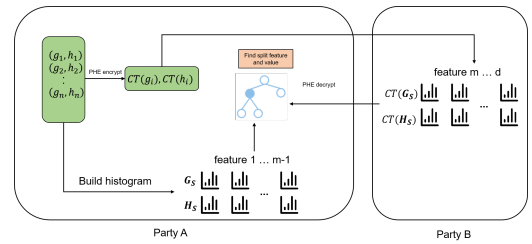


Figure 1: Training Process of SecureBoost [11].

Then, for each feature, find the best split of the samples that maximize the following function:

$$\text{Split_gain} = \frac{1}{2} \left[\frac{G_L^2}{H_L + \lambda} + \frac{G_R^2}{H_R + \lambda} - \frac{(G_L + G_R)^2}{H_L + H_R + \lambda} \right] - \gamma \quad (7)$$

Where λ is a hyperparameter, G_L and H_L , G_R and H_R are the sums of the gradients and Hessians of the data samples partitioned into the left and right branch based on the splitting points. At last, choose the feature with the maximal Split_gain for the current node and

split the samples accordingly. The weight of each leaf is computed by the following function:

$$\omega = -\frac{\sum_{i \in I_j} g_i}{\sum_{i \in I_j} h_i + \lambda} \quad (8)$$

In the context of vertical federated XGBoost, we assume there are two parties: Active Party A and Passive Party B, each holding $D_A = \{x_i^A, y_i\}$ and $D_B = \{x_i^B\}$ respectively, where $x_i^A \in \mathbb{R}^{m-1}$ and $x_i^B \in \mathbb{R}^{d-m+1}$. As described in Figure 1 proposed by Cheng et al. [11], Party A calculates g_i and h_i for all samples, encrypts them respectively using additively homomorphic encryption, and sends the ciphertexts $CT(g_i)$ and $CT(h_i)$ to all other participants to build histograms.

3.5 Matrix Vector Product based on Coefficient Encoding

Fully Homomorphic Encryption [16] gives the ability to compute arbitrary functions on the encrypted message. Contemporary FHE methods, based on the RLWE assumption, offer efficient ways to compute on encrypted polynomials [6, 7, 12]. Vectors or matrices can be represented as polynomials using SIMD encoding with discrete Fourier transform [33]. Notably, numerous studies have proposed a matrix-vector product protocol via SIMD encoding [17, 18, 20, 25], involving ciphertext rotation and multiplication.

An alternative paradigm is coefficient encoding, introduced by Yasuda et al. [39], which treats polynomial coefficients as vectors and interprets polynomial multiplication as a vector inner product. This method was further expanded by Duong et al. [14] into an efficient matrix-vector product over encrypted data, which requires just a single ciphertext multiplication. More recently, Huang et al. [19] applied this approach for secure inference on deep neural networks.

While the SIMD approach allows for chained homomorphic computation, it is relatively slow. Conversely, the coefficient encoding method is faster, but necessitates decryption after each operation. In the context of vertical federated learning, the decryption requirement of the coefficient encoding method is acceptable. Gives two encoding methods: $\pi^v : \mathbb{Z}_t^n \rightarrow \mathbb{R}_{N,t}$ and $\pi^w : \mathbb{Z}_t^{d \times n} \rightarrow \mathbb{R}_{N,t}$, where $d \times n \leq N$:

$$\begin{aligned} \mathbf{v} &= \pi^v(\mathbf{v}) : \mathbf{v}[j] = v[j], \text{ for } j \in \{0, 1, \dots, n-1\} \\ \mathbf{m} &= \pi^w(\mathbf{M}) : \mathbf{m}[i \cdot n + n - 1 - j] \\ &= \mathbf{M}[i, j], \text{ for } i \in \{0, 1, \dots, d-1\} \end{aligned} \quad (9)$$

given $\mathbf{v} = \pi^v(\mathbf{v})$, $\mathbf{m} = \pi^w(\mathbf{M})$, $\mathbf{u} = \mathbf{M}\mathbf{v}$ can be evaluated by $\mathbf{u} = \mathbf{m} \cdot \mathbf{v}$, where $\mathbf{u}[i \cdot n + n - 1] = \mathbf{u}[i]$, $i \in \{0, 1, \dots, d-1\}$.

The homomorphic encryption method employed in this study adheres to the BFV convention as outlined by Brakerski [6], where the message space is in $\mathbb{R}_{N,t}$. The secret key is defined as $sk = \mathbf{s}$, and the public key is defined as $pk = (\mathbf{b} = -\mathbf{a}\mathbf{s} + \mathbf{e}, \mathbf{a})$, where $\mathbf{a}$ is uniformly chosen from $\mathbb{R}_{N,q}$ and $\mathbf{e}$ is a random noise. The encryption of the message $\mathbf{m} \in \mathbb{R}_{N,t}$ is denoted as

$$\begin{aligned} CT(\mathbf{m}) &= (c_0, c_1) = BFV.enc(pk, \mathbf{m}) \\ &= (\mathbf{e}' + \lfloor q/t \cdot \mathbf{m} \rfloor, 0) + \mathbf{u} \cdot pk \in \mathbb{R}_{N,q}^2 \end{aligned} \quad (10)$$

Where $\mathbf{u}$ is a uniformly chosen polynomial from $\mathbb{R}_{N,q}$, $\mathbf{e}'$ is a random noise. The corresponding decryption is denoted as

$$\mathbf{m} = BFV.dec(sk, CT(\mathbf{m})) = \lfloor t/q \cdot (c_0 + c_1 \mathbf{s}) \rfloor \in \mathbb{R}_{N,t}. \quad (11)$$

The matrix vector product algorithm is described in Algorithm 2, where Alice and Bob can jointly compute $\mathbf{u} = \mathbf{M}\mathbf{v}$ with privately hold $\mathbf{M}$ and $\mathbf{v}$. By utilizing the coefficient encoding method, the matrix vector product is extremely fast.

3.6 Quantum-safe cryptography

Quantum-safe cryptography, also referred to as quantum-resistant or post-quantum cryptography, is a branch of cryptographic techniques aiming to develop next generation cryptographic systems capable of withstanding potential attacks relying on special quantum algorithms run on quantum computers [4]. For instance, Shor's algorithm [32] can efficiently factor large integers and compute discrete logarithms, which are security fundamentals to the operation of RSA, ECC (Elliptic Curve Cryptography), and Paillier [27], etc. These operations are integral to federated learning, thus, raising potential security concerns for many existing federated learning models in the upcoming quantum computing era.

Post-quantum cryptography seeks cryptographic algorithms believed to be resistant to quantum computer attacks. The mostly widely studied and promising approach to achieving quantum-safe property is the lattice-based encryption [30]. In this work, we construct a federated learning model that utilizes lattice-based Fully Homomorphic Encryption (FHE), with parameters specifically chosen to resist attacks from quantum computers [1]. This ensures the long-term security of our models even in the advent of practical quantum computing.

4 SCALABLE SECURE VERTICAL LR

Our proposed method for vertical federated linear regression (LR) follows a process akin to that outlined by [38]. In our method, we encode $\mathbf{z}$ using lines 1-3 of Algorithm 2, then encrypt it based on line 4 of Algorithm 2. For $\mathbf{B}$ (as referenced on line 7 of Algorithm 1), it computes $CT(g^B)$ utilizing lines 5-6 of 2]. By substituting certain steps — specifically lines 5-9 in Algorithm 1 — with the matrix-vector multiplication from Algorithm 2, we can easily derive a faster and more scalable secure vertical LR protocol without additively homomorphic encryption. We will discuss the performance details in the experimental section of this paper.

5 SECURE XGBOOST WITH COEFFICIENT ENCODING

5.1 Selection Methods for Splitting Points

In XGBoost [10], it uses Weighted Quantile Sketch to select candidate splitting points, and then constructs feature histograms based on these splitting points. However, when the passive party receives the first-order gradient and second-order hessian encrypted using the homomorphic encryption algorithm, there will be a high computational complexity. Therefore, in order to speed up the matrix multiplication using the coefficient encoding mentioned above, we use equal-frequency binning Algorithm 3 to select candidate splitting points for each feature from passive party. As a result, there are l bins and $(l-1)$ candidate splitting points for each feature of

Algorithm 2 Secure Matrix Vector Product

Input: $\mathbf{v} \in \mathbb{Z}_t^n$ from party Alice, $\mathbf{M} \in \mathbb{Z}_t^{d \times n}$ from party Bob.

Input: The common input is a BFV public key pk , a common partition size n_p, d_p , where $0 < n_p < n, 0 < d_p < d$ and $n_p d_p \leq N$, let $n' = \lceil n/n_p \rceil, d' = \lceil d/d_p \rceil$.

Output: $CT(\mathbf{u}_\beta) \in \mathbb{R}_{N,q}^2, \beta \in \{0, 1, \dots, d' - 1\}$, where $\mathbf{u} = \mathbf{M}\mathbf{v} \in \mathbb{Z}_t^d$ can be extracted from $\mathbf{u}_\beta$.

- 1: Alice pads the vector $\mathbf{v}$ to a length that's a multiple of n_p with zeros. She then divides the vector $\mathbf{v}$ into $\mathbf{v}_\alpha \in \mathbb{Z}_t^{n_p}$, where $\alpha \in \{0, 1, \dots, n' - 1\}$.
- 2: Bob pads matrix $\mathbf{M}$ to the shape of multiples of $d_p \times n_p$ with zeros, and partitions $\mathbf{M}$ into $\mathbf{M}_{\beta,\alpha} \in \mathbb{Z}_t^{d_p \times n_p}$, for $\beta \in \{0, 1, \dots, d' - 1\}, \alpha \in \{0, 1, \dots, n' - 1\}$.
- 3: Alice encodes the vector $\mathbf{v}_\alpha$ to $\mathbf{v}_\alpha = \pi^v(\mathbf{v}_\alpha)$, for $\alpha \in \{0, 1, \dots, n' - 1\}$.
- 4: Alice encrypts the polynomials $\mathbf{v}_\alpha$ as $CT(\mathbf{v}_\alpha) = BFV.enc(pk, \mathbf{v}_\alpha)$. Alice sends $CT(\mathbf{v}_\alpha)$ to Bob.
- 5: Bob encode the matrixes to get $\mathbf{m}_{\beta,\alpha} = \pi^w(\mathbf{M}_{\beta,\alpha}), \beta \in \{0, 1, \dots, d' - 1\}$.
- 6: Bob compute $CT(\mathbf{u}_\beta) = (c_{\beta,0}, c_{\beta,1}) = \sum_{\alpha \in [0, n' - 1]} (\mathbf{m}_{\beta,\alpha} \cdot CT(\mathbf{v}_\alpha)) \in \mathbb{R}_{N,q}^2$.
- 7: Note that $\mathbf{u}$ can be extracted from $\{\mathbf{u}_\beta\}$ with $\mathbf{u}[i] = \mathbf{u}_i \bmod d' [(i \bmod d')n + n - 1], i \in \{0, 1, \dots, d - 1\}$. Bob keeps the coefficient of $c_{\beta,0}[(i \bmod d')n + n - 1]$, and set other coefficients to zero.

passive party. As for active party A, it can use Weighted Quantile Sketch to select candidate splitting points as it has plain gradient and hessian.

5.2 Distributed Histogram Construction

As described in Algorithm 3, we use equal-frequency binning (line 4-28) to select candidate splitting points $s = [s_1, s_2, \dots, s_{l-1}]$ for each feature of the passive party and use Weighted Quantile Sketch for active the party respectively. As a result, the active party can easily use its candidate splitting points and plain gradient (hessian) to construct feature histograms without passive parties' help. But as for passive party, it can not use plain gradient (hessian) to construct feature histograms due to data privacy concerns [41, 42]. Based on SecureBoost [11], we use Coefficient Encoding as equation 9 and BFV encryption as equation 10 mentioned in section 3.5 to protect gradient and hessian instead of Partially Homomorphic Encryption (paillier [27], etc.). As mentioned above, the key for vertical federated xgboost model training is how can the passive party quickly build its own feature histograms without revealing the privacy of the gradient and hessian from the active party. To solve this, we first construct an indicator matrix $\mathbf{M} \in \{0, 1\}^{(l-1) \times n}$ for each feature $x = [x_1, x_2, \dots, x_n] \in X_B^{n \times (d-m+1)}$ from the passive party:

$$\mathbf{M} = \begin{pmatrix} b_{1s_1} & \dots & b_{ns_1} \\ b_{1s_2} & \dots & b_{ns_2} \\ \vdots & \vdots & \vdots \\ b_{1s_{l-1}} & \dots & b_{ns_{l-1}} \end{pmatrix}, \text{ where } b_{is_j} = \begin{cases} 1, & \text{if } x_i \leq s_j \\ 0, & \text{if } x_i > s_j \end{cases}, \quad (12)$$

Briefly, we build only a single feature histogram for active party and passive party using Algorithm 4. As described in Algorithm 4 (line 1), firstly, the active party A first preprocesses g and h using coefficient encoding as Algorithm 2 (line 1-3) and encrypts them according to Algorithm 2 (line 4). Subsequently, the passive party B computes $CT(G^{l-1})$ and $CT(H^{l-1})$ based on $CT(g)$ and $CT(h)$ using Algorithm 2 (line 5-6). Different from the existing method proposed by Cheng et al. [11], our proposed method does not require

the passive party to calculate the sum of the first-order gradient and the second-order Hessian for each bin separately according to Equation 6. Instead, this phase is completed with the assistance of the active party.

In line 3, the active party A decrypts $CT(G^{l-1})$ and $CT(H^{l-1})$ using equation 11 to obtain G^{l-1} and H^{l-1} . Here, each element $G_i \in G^{l-1}$ represents the sum of the second-order hessian of the feature $x = [x_1, x_2, \dots, x_n]$ less than or equal to s_j from passive party. Similarly, each element $H_i \in H^{l-1}$ represents the sum of the second-order hessian of the feature $x = [x_1, x_2, \dots, x_n]$ less than or equal to s_j from passive party. After that, the active party holds all the passive parties' data feature histograms and its own histograms. It can then use these histograms to calculate the *Split_gain* based on equation 7 to find the optimal splitting point.

5.3 Training

Based on the previous subsection, our proposed method is motivated by Coefficient Encoding as equation 9 and the secure matrix-vector product as in Algorithm 2 that allows the participants to determine the optimal splitting candidate securely. In general, our proposed method achieves equivalent privacy-preserving as SecureBoost in [11] using partially homomorphic encryption technique. Algorithm 5 describes the procedure of our proposed vertical federated XGBoost. There are $(k-1)(k \geq 2 \in \mathbb{Z}^+)$ passive parties and one active party. For simplicity, we assume each party (all passive and active parties) holds only a single feature.

The whole training process is depicted in Algorithm 5. Firstly (on line 2), all passive and active parties run Algorithm 3 to get candidate splitting points S^j for X^j . In the iterative tree building, active party computes $g, h \in \mathbb{R}^n$ based on equation 5. Notice that these gradients (hessians) need to be updated when a new tree is initialized, based on the prediction results $[\hat{y}_1, \hat{y}_2, \dots, \hat{y}_n]$ of the previous tree. We remark that this process only needs to be done once at the beginning of the iteration. Then, use its candidate split points and plain gradient (hessian) to construct feature histograms without passive parties' help. While, all passive parties run Algorithm 4

Algorithm 3 select candidate splitting points for a single feature

Input: $X_A \in R^{n \times 1}$ of active party; $X_B \in R^{n \times 1}$ of passive party;
 $\max_bin = l, n \gg l$.

Output: candidate split points

- 1: **Active party A:**
- 2: A computes candidate split points using Weighted Quantile Sketch Chen and Guestrin [10] for X_A .
- 3: **Active party B:**
- 4: $D = \text{SORTED}(X_B)$, $\text{bin_size} = \lfloor n/l \rfloor$, $\text{remain} = n - \text{bin_size} \times l$,
- 5: $\text{bin_size_all} = [\underbrace{\text{bin_size}, \dots, \text{bin_size}}_l], \text{bin_feature} = [\underbrace{[], \dots, []}_l]$
- 6: **if** $\text{remain} < 0$ **then**
- 7: $\text{bin_size_all}[l-1] += \text{remain}$
- 8: **else if** $\text{remain} > 0$ **then**
- 9: $\text{bin_size_all}[l-1] -= \text{remain}$
- 10: **end if**
- 11: $\text{start} = 0$
- 12: **for** $i = 1$ **to** l **do**
- 13: $\text{end} = \text{start} + \text{bin_size_all}[i]$, $\text{bin_feature}[i] = D[\text{start} : \text{end}]$,
 $\text{start} = \text{end}$.
- 14: **end for**
- 15: $\text{candidate} = \{\}$, $\text{acc_num} = 0$
- 16: **for** $i = 1$ **to** l **do**
- 17: $\text{sample_num} = \text{len}(\text{bin_feature}[i])$
- 18: $v_{\max} = D[\text{bin_feature}[i]][-1]$
- 19: $v_{\min} = D[\text{bin_feature}[i]][0]$
- 20: **if** $\text{sample_num} == 0$ **then**
- 21: $\text{candidate}[\text{acc_num} + \text{sample_num}] = v_{\max} / 2$
- 22: **else if** $i == (l - 1)$ **then**
- 23: $\text{candidate}[\text{acc_num}] += v_{\min} / 2$
- 24: **else**
- 25: $\text{candidate}[\text{acc_num}] += v_{\min} / 2$
- 26: $\text{candidate}[\text{acc_num} + \text{sample_num}] = v_{\max} / 2$
- 27: **end if**
- 28: $\text{acc_num} += \text{sample_num}$
- 29: **end for**
- 30: **Return** candidate

(line 2-3) to construct a complete feature histogram. After finding the best splitting feature (line 14), P_k tells the participant P_j holding this feature how the samples are split (line 15).

6 SECURITY ANALYSIS

It is critical to analyze the security aspects of our proposed Vertical Federated Learning (VFL) solutions for Logistic Regression (LR) and XGBoost, as privacy and data security form the bedrock of any federated learning system. Our security analysis focuses on two key aspects: comparing the security level with prior works and discussing the enhancement brought by post-quantum techniques.

6.1 Comparison with Prior Works

Our proposed LR and XGBoost methods have been designed as enhanced versions of the methods proposed in prior works proposed by Yang et al. [38] and Cheng et al. [11]. Specifically, we have improved upon the methods by incorporating a Matrix Vector Product based on Coefficient Encoding, which offers significant advantages in training efficiency and quantum security.

In terms of security, it is important to note that our proposed methods inherit the security attributes of the original methods. This is because the core processes of federated learning, particularly those related to privacy-preserving mechanisms, remain preserved in our methods. Therefore, at a minimum, our methods offer a security level that is comparable to the previously proposed methods.

6.2 Enhancement with Post-Quantum Techniques

Beyond maintaining the security level of prior works, our proposed methods have introduced the use of post-quantum techniques, which provides an additional layer of security. In the context of increasing capabilities of quantum computing, the potential vulnerabilities of classical cryptographic systems have become a growing concern. Quantum computers have the potential to solve certain computational problems, such as integer factorization and discrete logarithm, significantly faster than classical computers. These problems serve as the foundation of many existing cryptographic systems.

By utilizing lattice-based Fully Homomorphic Encryption (FHE) with parameters specifically chosen to resist quantum computer attacks, our proposed methods ensure long-term security of the federated learning models. This means our methods are not only secure under the existing computational capabilities but are also designed to withstand potential future threats posed by quantum computing.

In summary, our proposed LR and XGBoost methods for VFL offer a high level of security. They maintain, at the least, the same level of security as prior works, while also enhancing the security attributes to be quantum-resistant. This forward-looking approach to security ensures the applicability and resilience of our methods in the face of evolving computational landscapes.

7 EXPERIMENTS

In this section, we evaluate the performance of our proposed methods in section 4 and 5.

7.1 Experiment Setting

To begin with, the experimental setup is introduced below.

Environment. The experiments are conducted by five 4-cpu-core 8GB-memory Linux machines with network bandwidth of 25000Mb/s.

Datasets. As listed in Table 1, we conduct the experiments on four datasets, which contain a relatively large amount of data and features. All these datasets are randomly generated using `sklearn1.datasets.make_classification`. We randomly split 70% and

¹<https://scikitlearn.org/>

Algorithm 4 construct histogram for a single feature

Input: active party A has $g, h \in R^n$, candidate split points s_A ; passive party B has indicator matrix $\mathbf{M} \in \{0, 1\}^{(l-1) \times n}$

Output: feature histogram for active and passive party respectively

- 1: **A:** builds histogram using plain g and h based on s_A as Figure 1 does; Then A preprocesses g and h using coefficient encoding as Algorithm 2 (line 1-3) and encrypts them to get $CT(g)$ and $CT(h)$ using Algorithm 2 (line 4) and sends them to B.
 - 2: **B:** computes $CT(G^{l-1})$ and $CT(H^{l-1})$ based on $CT(g)$ and $CT(h)$ using Algorithm 2, and sends $CT(G^{l-1})$ and $CT(H^{l-1})$ to A; Additionally, B constructs l lists containing bin index and the number of samples in each bin, and sends them to A.
 - 3: **A:** decrypts $CT(G^{l-1})$ and $CT(H^{l-1})$ to get G^{l-1} and H^{l-1} . Then A builds a complete histogram for the passive party based on G^{l-1} and H^{l-1} .
 - 4: **Return** hist
-

Algorithm 5 the procedure of our proposed vertical federated XGBoost

Input: the active party P_k has $X^k \in R^{n \times 1}$, $y^{n \times 1} \in \{0, 1\}$; each passive party $P_i (i \in [1, k-1])$ has $X^i \in R^{n \times 1}$; $\text{max_bin} = l$, $n \gg l$.

Output: T decision trees

- 1: **for** $j = 1$ **to** k **do**
 - 2: run Algorithm 3 to get candidate splitting points S^j for X^j
 {for each party}
 - 3: **end for**
 - 4: active party P_k initializes $[\hat{y}_1, \hat{y}_2 \dots \hat{y}_n]$ with random values.
 - 5: **# start to build trees**
 - 6: **for** $t = 1$ **to** T **do**
 - 7: P_k computes $g, h \in R^n$ based on equation 5
 - 8: **for** each node in current tree **do**
 - 9: $HIST = []$
 - 10: **for** $j = 1$ **to** k **do**
 - 11: run Algorithm 4 to get $hist^k$ *{for each party}*
 - 12: $HIST.append(hist^k)$
 - 13: **end for**
 - 14: active party find maximal score and split feature and split bin_index based on $HIST$ using equation 7
 - 15: Split the sample id of the current node
 - 16: **end for**
 - 17: P_k updates $[\hat{y}_1, \hat{y}_2 \dots \hat{y}_n]$ of the leaf nodes based on the equation 8.
 - 18: **end for**
 - 19: **Return** T decision trees
-

30% as training and validation sets, respectively. We use synthetic1 and synthetic3 to compare the performance of different methods under different feature number. Besides, We use synthetic2 and synthetic4 to compare the performance of different methods under different data amounts. And we use synthetic3 and randomly split 100 features into different subsets on average and assign them to different parties to verify the scalability of our proposed algorithms.

REMARK: The reason why we only use synthetic datasets for experiments is that the key training processes of our proposed algorithms are not changed compared to existing algorithms, except for changing the encryption algorithm. Therefore, using these datasets

Table 1: Description of datasets for Experiments

Dataset	#Sample_num	#Feature(A/B)
synthetic1	150K	5/5
synthetic2	150k	10/13
synthetic3	150K	50/50
synthetic4	300K	10/13

can completely achieve the purpose of algorithm performance comparison.

Competitors. We evaluate our proposed vertical federated Logistic Regression by comparing the model accuracy and the time consumption with lr_phe [38] and a baseline method, in which we use tenseal [3] as the encryption method. And we evaluate our proposed vertical federated XGBoost with SecureBoost [11] and xgb_batch [36].

We set the number of trees as $T = 10$, $\text{max_depth} = 3$ for XGBoost and $\text{max_iters} = 10$, $\text{batch_size} = 20000$ for Logistic Regression. We conduct [11, 36, 38] using Paillier [27], in which we set paillier key_length=1024. All experiments were repeated 5 times and the averages are reported.

7.2 Performance under different data amounts

We first evaluate the utility of our proposed vertical LR with different number of batch_sizes on synthetic2. Detailed results are provided in Table 2. As shown in Table 2, the larger the batch_size we set, the more obvious the performance improvement will be. The result shows that the use of this method can significantly increase the computing efficiency of complex algorithms.

Then, we fix the batch_size = 20000 and run the three methods. Figure 2 and 3 depict validation loss of the all methods on synthetic2 and synthetic4. As can be seen from figure 2a and figure 2b, although different methods are used to implement vertical federated LR, the loss after each iteration is almost the same under the same parameter settings. Table 3a evaluates the time consumption and accuracy of different methods for vertical federated LR. All methods achieve almost the same AUC. However, in terms of time consumption, our proposed method is much superior to other methods under different data amounts, which improves performance by 10~30x compared to existing method [38]. Compared with the method using fully homomorphic encryption tenseal [3], the performance is improved by 5~40x. Additionally, by comparing the

results on synthetic2 and synthetic4, when the amount of data is doubled, the time-consuming growth rate of the three algorithms is almost the same.

Table 2: Utility of our proposed vertical LR with different number of batch_sizes on synthetic2

batch_size	AUC	Time(s)
5k	0.9940	150
20k	0.9938	67
50k	0.9938	40
100k	0.9939	29

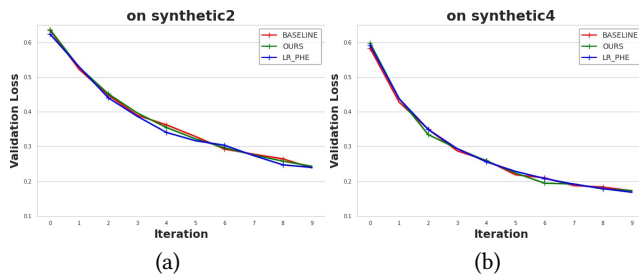


Figure 2: Validation Loss of the three methods of vertical LR on synthetic2 and synthetic4. The figures show the loss under different data amounts.

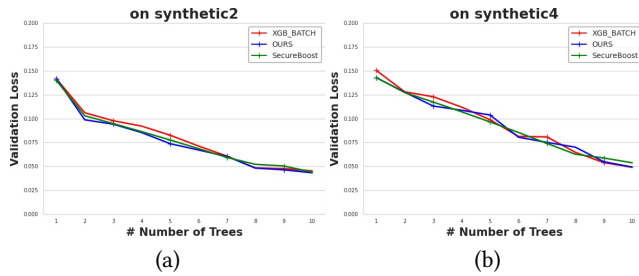


Figure 3: Validation Loss of the three methods of vertical XGBoost on synthetic2 and synthetic4. The figures show the loss under different data amounts.

Similarly, figure 3a and figure 3b show the performance of different methods used in vertical federated XGBoost. Under the same parameter settings, the loss after each iteration is also almost the same. Table 3b evaluates the time consumption and accuracy of different methods for vertical federated XGBoost. And all methods achieve almost the same AUC. Similarly, in terms of time consumption, our proposed method improves performance by 1.5x compared to SecureBoost [11] under different data amounts, achieving approximately the same performance as [36] or even better.

Table 3: Performance of all methods under different data amounts

Dataset	Method	Time(s)	AUC
synthetic2	baseline	2324	0.9939
	ours	67	0.9938
	lr_phe [38]	1391	0.9938
synthetic4	baseline	4656	0.9947
	ours	224	0.9947
	lr_phe [38]	3184	0.9948

(a) vertical LR on synthetic2 and synthetic4

Dataset	Method	Time(s)	AUC
synthetic2	ours	489	0.9940
	SecureBoost [11]	792	0.9939
	xgb_batch [36]	507	0.9939
synthetic4	ours	935	0.9948
	SecureBoost [11]	1587	0.9949
	xgb_batch [36]	996	0.9950

(b) vertical XGBoost on synthetic2 and synthetic4

7.3 Performance under different feature number

Figure 4 and 5 depict validation loss of all methods on Synthetic1 and Synthetic3. Coincidentally, similar conclusions to Section 7.2 can be reached under varying numbers of features. In general, compared with other methods, ours significantly accelerate the training process whether it is vertical federated XGBoost or vertical federated LR. In addition, we figured out an interesting result in vertical federated LR, that is, with the same amount of data, there is almost no difference in the performance of the method implemented using teneal [3] when increasing the feature number d ($d \ll \text{batch_size}$). The reason lies in the fact that the matrix vector product used in teneal pads the matrix into a square matrix of at least $\text{batch_size} \times \text{batch_size}$ shape, whereas N is the polynomial degree, its performance does not scale with d when $d \ll \text{batch_size}$. This also indirectly explains that when the amount of data is the same and the feature number increases, the performance of [38] implemented based on paillier [27] will gradually become inferior to the vertical LR implemented by teneal.

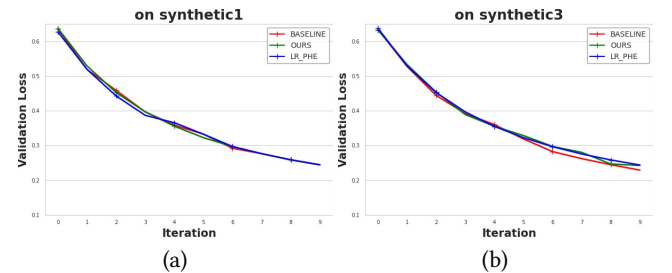


Figure 4: Validation Loss of the three methods of vertical LR on synthetic1 and synthetic3. The figures show the loss under different feature number.

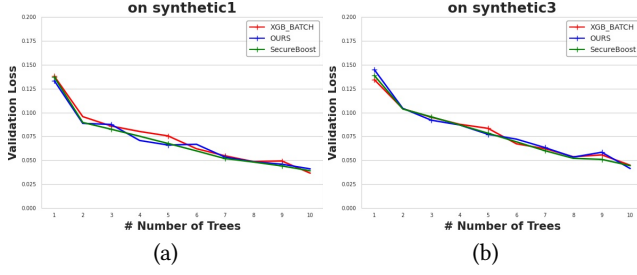


Figure 5: Validation Loss of the three methods of vertical XGBoost on synthetic1 and synthetic3. The figures show the loss under different feature number.

Table 4: Performance of all methods under different feature number

Dataset	Method	Time(s)	AUC
synthetic1	baseline	2343	0.9950
	ours	50	0.9951
	lr_phe [38]	1308	0.9950
synthetic3	baseline	2340	0.9935
	ours	189	0.9936
	lr_phe [38]	3245	0.9936
(a) vertical LR on synthetic1 and synthetic3			
Dataset	Method	Time(s)	AUC
synthetic1	ours	385	0.9956
	SecureBoost [11]	602	0.9957
	xgb_batch [36]	407	0.9956
synthetic3	ours	1547	0.9937
	SecureBoost [11]	2152	0.9939
	xgb_batch [36]	1694	0.9938
(b) vertical XGBoost on synthetic1 and synthetic3			

7.4 Scalability

Finally, we empirically explore the scalability of our two proposed methods when the number of parties increases. Although our discussion focuses on the two-party scenario, it can be easily generalized to the cases of multiple parties, i.e., there are two or more parties that work as **Passive Party (PP)** and contribute their features to train the model of **Active Party (AP)**. Thus, we evaluate all methods using 3-5 parties². The dataset synthetic3 is used in the experiments. we randomly split the features into different subsets on average and assign them to different parties. For instance, each party owns 20 features for the dataset synthetic3. The results are shown in Table 5. In general, when there are more parties, all methods achieve higher AUC since the total amount of features increases. Besides, the two algorithms we proposed still perform an order of magnitude lead regardless of the number of parties. For the training time growth, although when the number of passive parties increases, the active party takes more time to encrypt and

²Unlike horizontal federated learning, the number of parties is always 2-3 in most vertical federated learning scenarios, and rarely exceeds 5, so we conduct experiments with no more than five parties.

Table 5: Performance of all methods when there are different number of parties

#Parties & #Feature	Method	Time(s)	AUC
2	baseline	2430	0.7594
	ours	123	0.7605
	lr_phe	1940	0.7601
3	baseline	3828	0.8949
	ours	190	0.8939
	lr_phe	2248	0.8951
4	baseline	5745	0.9858
	ours	255	0.9861
	lr_phe	2586	0.9860
5	baseline	7491	0.9936
	ours	350	0.9935
	lr_phe	2963	0.9938

(a) vertical LR on synthetic3

#Parties & #Feature	Method	Time(s)	AUC
2	ours	765	0.8339
	SecureBoost	1094	0.8338
	xgb_batch	807	0.8345
3	ours	897	0.9211
	SecureBoost	1336	0.9215
	xgb_batch	994	0.9218
4	ours	1037	0.9914
	SecureBoost	1577	0.9908
	xgb_batch	1198	0.9912
5	ours	1368	0.9938
	SecureBoost	1819	0.9937
	xgb_batch	1452	0.9939

(b) vertical XGBoost on synthetic3

decrypt data. Nevertheless, our proposed two methods are able to scale to more parties within a reasonable time increment given the improvement of model ability.

8 CONCLUSION AND FUTURE WORK

In this paper, we studied vertical federated learning, in particular, LR and XGBoost modeling. To address the inefficiency and quantum security risks resulted by conventional privacy-preserving methods, we proposed a novel strategy, HQsFL, for vertical federated learning using Matrix Vector Product based on Coefficient Encoding. Numerical experiments demonstrated that our proposed method can maintain the same level of AUC with conventional methods while greatly improving the training efficiency and achieving important security property of quantum-computing resistance that may render the system long-term safety in quantum era.

9 ACKNOWLEDGMENT

This research was supported in part by the Ministry of Science and Technology of the People's Republic of China through the project on Cyberspace Security Governance (2023YFB3106501), by the National Natural Science Foundation of China (62227805, 62072398, 62132008), by the Natural Science Foundation of Jiangsu Province (BK20220075) and by the Fok Ying-Tung Education Foundation for Young Teachers in the Higher Education Institutions of China (No.20193218210004).

REFERENCES

- [1] Martin R Albrecht, Rachel Player, and Sam Scott. 2015. On the concrete hardness of learning with errors. *Journal of Mathematical Cryptology* 9, 3 (2015), 169–203.
- [2] Yoshinori Aono, Takuya Hayashi, Le Trieu Phong, and Lihua Wang. 2016. Privacy-preserving logistic regression with distributed data sources via homomorphic encryption. *IEEE TRANSACTIONS on Information and Systems* 99, 8 (2016), 2079–2089.
- [3] Ayoub Benaissa, Bilal Retiat, Bogdan Cebere, and Alaa Eddine Belfedhal. 2021. TenSEAL: A Library for Encrypted Tensor Operations Using Homomorphic Encryption. *arXiv:2104.03152 [cs.CR]*
- [4] Daniel J Bernstein and Tanja Lange. 2017. Post-quantum cryptography. *Nature* 549, 7671 (2017), 188–194.
- [5] Keith Bonawitz, Vladimir Ivanov, Ben Kreuter, Antonio Marcedone, H Brendan McMahan, Sarvar Patel, Daniel Ramage, Aaron Segal, and Karn Seth. 2017. Practical secure aggregation for privacy-preserving machine learning. In *proceedings of the 2017 ACM SIGSAC Conference on Computer and Communications Security*. 1175–1191.
- [6] Zvika Brakerski. 2012. Fully homomorphic encryption without modulus switching from classical GapSVP. In *Annual Cryptology Conference*. Springer, 868–886.
- [7] Zvika Brakerski, Craig Gentry, and Vinod Vaikuntanathan. 2014. (Leveled) fully homomorphic encryption without bootstrapping. *ACM Transactions on Computation Theory (TOCT)* 6, 3 (2014), 1–36.
- [8] Chaochao Chen, Jun Zhou, Li Wang, Xibin Wu, Wenjing Fang, Jin Tan, Lei Wang, Alex X Liu, Hao Wang, and Cheng Hong. 2021. When homomorphic encryption marries secret sharing: Secure large-scale sparse logistic regression and applications in risk control. In *Proceedings of the 27th ACM SIGKDD Conference on Knowledge Discovery & Data Mining*. 2652–2662.
- [9] Tianyu Chen, Hangbo Bao, Shaohan Huang, Li Dong, Binxing Jiao, Daxin Jiang, Haoyi Zhou, Jianxin Li, and Furu Wei. 2022. The-x: Privacy-preserving transformer inference with homomorphic encryption. *arXiv preprint arXiv:2206.00216* (2022).
- [10] Tianqi Chen and Carlos Guestrin. 2016. Xgboost: A scalable tree boosting system. In *Proceedings of the 22nd acm sigkdd international conference on knowledge discovery and data mining*. 785–794.
- [11] Kewei Cheng, Tao Fan, Yilun Jin, Yang Liu, Tianjian Chen, Dimitrios Papadopoulos, and Qiang Yang. 2021. Secureboost: A lossless federated learning framework. *IEEE Intelligent Systems* 36, 6 (2021), 87–98.
- [12] Jung Hee Cheon, Andrey Kim, Miran Kim, and Yongsoo Song. 2017. Homomorphic encryption for arithmetic of approximate numbers. In *Advances in Cryptology—ASIACRYPT 2017: 23rd International Conference on the Theory and Applications of Cryptology and Information Security, Hong Kong, China, December 3–7, 2017, Proceedings, Part I* 23. Springer, 409–437.
- [13] Ye Dong, Wen-jie Lu, Yancheng Zheng, Haoqi Wu, Derun Zhao, Jin Tan, Zhicong Huang, Cheng Hong, Tao Wei, and Wenguang Cheng. 2023. PUMA: Secure Inference of LLaMA-7B in Five Minutes. *arXiv preprint arXiv:2307.12533* (2023).
- [14] Dung Hoang Duong, Pradeep Kumar Mishra, and Masaya Yasuda. 2016. Efficient secure matrix multiplication over LWE-based homomorphic encryption. *Tatra mountains mathematical publications* 67, 1 (2016), 69–83.
- [15] Mingxuan Fan, Yilun Jin, Liu Yang, Zhenghang Ren, and Kai Chen. 2023. VER-TICES: Efficient Two-Party Vertical Federated Linear Model with TTP-aided Secret Sharing. *arXiv preprint arXiv:2306.16139* (2023).
- [16] Craig Gentry. 2009. Fully homomorphic encryption using ideal lattices. In *Proceedings of the forty-first annual ACM symposium on Theory of computing*. 169–178.
- [17] Shai Halevi and Victor Shoup. 2014. Algorithms in helib. In *Advances in Cryptology—CRYPTO 2014: 34th Annual Cryptology Conference, Santa Barbara, CA, USA, August 17–21, 2014, Proceedings, Part I* 34. Springer, 554–571.
- [18] Shai Halevi and Victor Shoup. 2018. Faster homomorphic linear transformations in HELib. In *Annual International Cryptology Conference*. Springer, 93–120.
- [19] Zhicong Huang, Wen-jie Lu, Cheng Hong, and Jiansheng Ding. 2022. Cheetah: Lean and fast secure {two-party} deep neural network inference. In *31st USENIX Security Symposium (USENIX Security 22)*. 809–826.
- [20] Xiaoqian Jiang, Miran Kim, Kristin Lauter, and Yongsoo Song. 2018. Secure outsourced matrix computation and application to neural networks. In *Proceedings of the 2018 ACM SIGSAC conference on computer and communications security*. 1209–1222.
- [21] Rakpong Kaewpuang, Minrui Xu, Dusit Niyato, Han Yu, Zehui Xiong, and Xuemin Sherman Shen. 2023. Adaptive Resource Allocation in Quantum Key Distribution (QKD) for Federated Learning. In *2023 International Conference on Computing, Networking and Communications (ICNC)*. IEEE, 71–76.
- [22] Peter Kairouz, H Brendan McMahan, Brendan Avent, Aurélien Bellet, Mehdi Bennis, Arjun Nitin Bhagoji, Kallista Bonawitz, Zachary Charles, Graham Cormode, Rachel Cummings, et al. 2021. Advances and open problems in federated learning. *Foundations and Trends® in Machine Learning* 14, 1–2 (2021), 1–210.
- [23] Harashta Tatimma Larasati, Muhammad Firdaus, and Howon Kim. 2022. Quantum Federated Learning: Remarks and Challenges. In *2022 IEEE 9th International Conference on Cyber Security and Cloud Computing (CSCloud)/2022 IEEE 8th International Conference on Edge Computing and Scalable Cloud (EdgeCom)*. IEEE, 1–5.
- [24] Tian Li, Anit Kumar Sahu, Ameet Talwalkar, and Virginia Smith. 2020. Federated learning: Challenges, methods, and future directions. *IEEE signal processing magazine* 37, 3 (2020), 50–60.
- [25] Wen-jie Lu, Zhicong Huang, Cheng Hong, Yiping Ma, and Hunter Qu. 2021. PEGASUS: bridging polynomial and non-polynomial evaluations in homomorphic encryption. In *2021 IEEE Symposium on Security and Privacy (SP)*. IEEE, 1057–1073.
- [26] Brendan McMahan, Eider Moore, Daniel Ramage, Seth Hampson, and Blake Aguera y Arcas. 2017. Communication-efficient learning of deep networks from decentralized data. In *Artificial intelligence and statistics*. PMLR, 1273–1282.
- [27] Pascal Paillier. 1999. Public-key cryptosystems based on composite degree residuosity classes. In *International conference on the theory and applications of cryptographic techniques*. Springer, 223–238.
- [28] Xiaoxuan Pan, Xi Cao, Weiting Wang, Ziyue Hua, Weizhou Cai, Xuegang Li, Haiyan Wang, Jiaqi Hu, Yipu Song, Dong-Ling Deng, et al. 2023. Experimental quantum end-to-end learning on a superconducting processor. *npi Quantum Information* 9, 1 (2023), 18.
- [29] Jun Qi. 2022. Federated quantum natural gradient descent for quantum federated learning. *arXiv preprint arXiv:2209.00564* (2022).
- [30] Oded Regev. 2004. Quantum computation and lattice problems. *SIAM J. Comput.* 33, 3 (2004), 738–760.
- [31] Zhaosen Shi, Zeyu Yang, Alzubair Hassan, Fagen Li, and Xuyang Ding. 2023. A privacy preserving federated learning scheme using homomorphic encryption and secret sharing. *Telecommunication Systems* 82, 3 (2023), 419–433.
- [32] Peter W Shor. 1999. Polynomial-time algorithms for prime factorization and discrete logarithms on a quantum computer. *SIAM review* 41, 2 (1999), 303–332.
- [33] Nigel P Smart and Frederik Vercauteren. 2014. Fully homomorphic SIMD operations. *Designs, codes and cryptography* 71 (2014), 57–81.
- [34] Qi Xia, Zeyi Tao, and Qun Li. 2021. Defending against byzantine attacks in quantum federated learning. In *2021 17th International Conference on Mobility, Sensing and Networking (MSN)*. IEEE, 145–152.
- [35] Rongxin Xu, Shiva Raj Pokhrel, Qijun Lan, and Gang Li. 2023. Post Quantum Secure Blockchain-based Federated Learning for Mobile Edge Computing. *arXiv preprint arXiv:2302.13258* (2023).
- [36] Wuxing Xu, Hao Fan, Kaixin Li, and Kai Yang. 2021. Efficient batch homomorphic encryption for vertically federated xgboost. *arXiv preprint arXiv:2112.04261* (2021).
- [37] Qiang Yang, Yang Liu, Tianjian Chen, and Yongxin Tong. 2019. Federated machine learning: Concept and applications. *ACM Transactions on Intelligent Systems and Technology (TIST)* 10, 2 (2019), 1–19.
- [38] Shengwen Yang, Bing Ren, Xuhui Zhou, and Liping Liu. 2019. Parallel distributed logistic regression for vertical federated learning without third-party coordinator. *arXiv preprint arXiv:1911.09824* (2019).
- [39] Masaya Yasuda, Takeshi Shimoyama, Jun Kogure, Kazuhiro Yokoyama, and Takeshi Koshihara. 2015. New packing method in somewhat homomorphic encryption and its applications. *Security and Communication Networks* 8, 13 (2015), 2194–2213.
- [40] Yichi Zhang, Chao Zhang, Cai Zhang, Lixin Fan, Bei Zeng, and Qiang Yang. 2022. Federated Learning with Quantum Secure Aggregation. *arXiv preprint arXiv:2207.07444* (2022).
- [41] Bo Zhao, Konda Reddy Mopuri, and Hakan Bilen. 2020. idlg: Improved deep leakage from gradients. *arXiv preprint arXiv:2001.02610* (2020).
- [42] Ligeng Zhu, Zhijian Liu, and Song Han. 2019. Deep leakage from gradients. *Advances in neural information processing systems* 32 (2019).